

BỘ GIÁO DỤC & ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN TỰ ĐỘNG ĐIỀU KHIỂN

-----Δ-----



XỬ LÝ ẢNH

**Nhận diện và phân loại phương tiện giao
thông từ video và hình ảnh theo thời gian
thực**

GVHD : TS. Dương Minh Thiện

SVTH : Nguyễn Công Minh 21151128

SVTH : Võ Sỹ Bình 21151445

Nhóm : 7

TP. HỒ CHÍ MINH – 12/2024

LỜI CẢM ƠN

Trước hết, em xin chân thành gửi lời cảm ơn sâu sắc tới thầy Dương Minh Thiện, giảng viên bộ môn Xử lý ảnh. Trong suốt quá trình học tập và nghiên cứu, thầy đã không chỉ truyền đạt cho em những kiến thức chuyên môn quý báu mà còn luôn là nguồn động viên lớn lao, giúp em vượt qua những khó khăn trong học tập và nghiên cứu.

Những bài giảng của thầy không chỉ đầy đủ và dễ hiểu mà còn rất thực tiễn, mang lại cho em cái nhìn sâu sắc về các phương pháp xử lý ảnh hiện đại và các ứng dụng của chúng trong nhiều lĩnh vực khác nhau. Thầy luôn dành thời gian để giải đáp mọi thắc mắc, hướng dẫn em và các bạn học sinh trong việc áp dụng lý thuyết vào thực tiễn một cách hiệu quả nhất.

Em cảm ơn thầy vì sự nhiệt tình và tâm huyết trong công tác giảng dạy, cũng như những lời khuyên quý giá mà thầy đã chia sẻ trong suốt khóa học. Những kiến thức và kỹ năng em học được từ thầy không chỉ giúp ích cho việc hoàn thành tiểu luận này mà còn sẽ theo em trong suốt quá trình học tập và phát triển nghề nghiệp sau này.

Em xin cảm ơn thầy một lần nữa vì sự tận tâm và sự quan tâm đến sự nghiệp học tập của từng sinh viên. Em chúc thầy luôn khỏe mạnh, hạnh phúc và tiếp tục có những đóng góp lớn lao cho ngành giáo dục cũng như các thế hệ học trò.

Thành phố Hồ Chí Minh, ngày 18 tháng 11 năm 2024

Nhóm thực hiện đề tài

MỤC LỤC

LỜI CẢM ƠN.....	i
MỤC LỤC	ii
DANH SÁCH HÌNH ẢNH	iv
DANH SÁCH BẢNG.....	v
CHƯƠNG 1. GIỚI THIỆU.....	1
1.1 Bối cảnh	1
1.2 Tại sao chọn YOLO v8?	2
1.3 Phạm vi thực hiện	2
1.4 Mục tiêu và đóng góp	3
1.5 Ý nghĩa thực tiễn.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	4
2.1 Tổng quan về YOLO	4
2.1.1 Đặc điểm nổi bật của YOLOv8.....	4
2.1.2 Tại sao sử dụng YOLOv8	6
2.2 Output của YOLO.....	8
2.3 Kiến trúc Mạng (Network Architecture)	8
2.4 Dự báo trên nhiều feature map trong YOLOv8.....	9
2.5 Anchor box	10
2.6 Hàm mất mát (Loss Function)	11
2.6.1 Classification Loss	11
2.6.2 Localization Loss.....	11
2.6.3 Confidence Loss	12
2.7 Dự báo bounding box	13
2.8 Non-Maximum Suppression (NMS)	14
2.9 Huấn Luyện và Tối Ưu Hóa	15
2.10 Phân loại YOLOv8	15

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	18
3.1 Tập dữ liệu chuẩn bị cho.....	18
3.2 Quá trình huấn luyện	20
3.3 Tiến hành huấn luyện.....	21
3.4 Đánh giá mô hình huấn luyện.....	24
3.4.1 Đánh giá chi tiết	24
3.4.2 Ưu điểm nhược điểm.....	25
CHƯƠNG 4. KẾT QUẢ.....	26
4.1 Kết quả huấn luyện	26
4.2 Kết quả kiểm tra trên ảnh thực tế.....	26
4.3 Đánh giá hiệu quả và hạn chế	28
4.4 Đề xuất cải thiện	29
PHỤ LỤC	30
TÀI LIỆU THAM KHẢO	34

DANH SÁCH HÌNH ẢNH

Hình 1: Hình ảnh giao thông việt nam	1
Hình 2: Yolo v8	2
Hình 3: Kiến trúc YOLO và cơ chế hoạt động.....	5
Hình 4: Các layer trong mạng darknet-53.	6
Hình 5: Chia ảnh đầu vào ra các ô.....	7
Hình 6: ví dụ output của YOLO là một véc tơ concatenate của 3 bounding boxes ...	8
Hình 7: Kiến trúc mạng (Network Architecture)	9
Hình 8: Output là 3 feature maps có kích thước lần lượt là 80x80, 40x40 và 20x20	10
Hình 9: Xác định anchor box cho một vật thể.....	11
Hình 10: Hình ảnh dự đoán bounding box	13
Hình 11: Từ 3 bounding box ban đầu cùng bao quanh chiếc xe đã giảm xuống còn một bounding box cuối cùng.	15
Hình 12: Chuẩn hóa class cho các lớp đối tượng	18
Hình 13: Sau khi dán chúng ta sẽ có 2 file ảnh và label.....	20
Hình 14: Đưa ảnh và file và label vào file coco128 để tiến hành huấn luyện.....	20
Hình 15: Lệnh chạy huấn luyện.....	21
Hình 16: Quá trình huấn luyện từ 1/60 đến 7/60.....	22
Hình 17: Kết quả của quá trình huấn luyện đầu tiên	23
Hình 18: Nhận diện trường hợp xe ít, buổi sáng	27
Hình 19: Nhận diện trường hợp xe ít, buổi tối	27
Hình 20: Trường hợp xe nhiều, buổi sáng.....	28
Hình 21: Trường hợp xe nhiều, buổi tối.....	28

DANH SÁCH BẢNG

Bảng 1: Phân loại mô hình	15
Bảng 2: Train lần 2	23
Bảng 3: Train lần 3	23
Bảng 4: Train lần 4	24

CHƯƠNG 1. GIỚI THIỆU

1.1 Bối cảnh

Trong bối cảnh đô thị hóa ngày càng phát triển, vấn đề giao thông trở thành một thách thức lớn đối với các thành phố trên toàn thế giới. Việc giám sát và quản lý giao thông không chỉ yêu cầu lực lượng nhân sự lớn mà còn cần các hệ thống tự động hóa hiệu quả để hỗ trợ:



Hình 1: Hình ảnh giao thông việt nam

- **Giảm thiểu ùn tắc giao thông:** Việc nhận diện và phân tích các loại phương tiện lưu thông trên đường giúp cơ quan chức năng điều chỉnh luồng xe và đưa ra các giải pháp giảm kẹt xe.
- **Hỗ trợ giám sát an ninh giao thông:** Phát hiện và theo dõi phương tiện có thể được ứng dụng trong việc kiểm tra phương tiện vi phạm hoặc theo dõi các hành vi bất thường.
- **Thu thập dữ liệu phân tích:** Tích hợp hệ thống AI để phát hiện và theo dõi đối tượng hỗ trợ thu thập dữ liệu quan trọng, như mật độ xe qua từng thời điểm, loại phương tiện phổ biến, và tốc độ trung bình.

CHƯƠNG 2.

Phương pháp truyền thống dựa trên cảm biến vật lý hoặc lực lượng con người gặp nhiều hạn chế về chi phí, độ chính xác và khả năng mở rộng. Do đó, công nghệ thị giác máy tính (computer vision) đang được triển khai mạnh mẽ trong lĩnh vực giao thông nhằm tự động hóa các nhiệm vụ giám sát.

1.2 Tại sao chọn YOLO v8?

YOLO (You Only Look Once) là một thuật toán phát hiện đối tượng mạnh mẽ và hiệu quả được thiết kế đặc biệt cho các ứng dụng thời gian thực. Phiên bản YOLO v8, được cải tiến bởi Ultralytics, sở hữu các ưu điểm vượt trội như:

- **Độ chính xác:** Khả năng nhận diện tốt các đối tượng nhỏ hoặc chồng lấn.
- **Tốc độ xử lý nhanh:** Đáp ứng yêu cầu thời gian thực, rất phù hợp cho các ứng dụng giám sát giao thông.
- **Tính linh hoạt:** Dễ dàng huấn luyện lại (fine-tuning) trên các tập dữ liệu cụ thể, chẳng hạn như tập dữ liệu phương tiện giao thông.



Hình 2: Yolo v8

1.3 Phạm vi thực hiện

Đề tài tập trung vào phát hiện và theo dõi phương tiện giao thông, gồm bốn loại chính:

1. Xe máy
2. Ô tô

- 3. Xe tải
- 4. Xe bus

1.4 Mục tiêu và đóng góp

Mục tiêu:

- Xây dựng một hệ thống phát hiện và theo dõi phương tiện giao thông tự động, ứng dụng trong các video thời gian thực hoặc video giao thông đã ghi sẵn.
- Tối ưu hóa độ chính xác và tốc độ xử lý để đáp ứng yêu cầu thời gian thực.

Đóng góp:

- Nghiên cứu và ứng dụng YOLO v8 cho phát hiện đối tượng giao thông.
- Tích hợp hệ thống theo dõi để giám sát liên tục các phương tiện qua từng khung hình.
- Đánh giá hiệu quả của thuật toán trong điều kiện thực tế, bao gồm mật độ giao thông cao, ánh sáng yếu, và che khuất.

1.5 Ý nghĩa thực tiễn

Hỗ trợ các cơ quan quản lý giao thông xây dựng hệ thống giám sát thông minh.

Ứng dụng trong phân tích dữ liệu giao thông để tối ưu hóa luồng xe và nâng cao an toàn giao thông.

Góp phần thúc đẩy nghiên cứu và phát triển các hệ thống thị giác máy tính ứng dụng trong đời sống thực tế.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Trong những năm gần đây, Object Detection đã trở thành một trong những lĩnh vực quan trọng nhất trong nghiên cứu và ứng dụng Deep Learning, nhờ vào khả năng ứng dụng rộng rãi và hiệu suất vượt trội trong thực tế. Các thuật toán hiện đại như YOLO (You Only Look Once) và SSD (Single Shot Multibox Detector) đã đạt được sự cân bằng giữa tốc độ và độ chính xác, giúp các ứng dụng phát hiện đối tượng hoạt động gần như thời gian thực. Trong đó, phiên bản YOLO v8 là một bước cải tiến đáng kể, cung cấp hiệu suất cao, được tối ưu hóa để triển khai trên nhiều nền tảng khác nhau, từ hệ thống nhúng đến các môi trường điện toán đám mây.

2.1 Tổng quan về YOLO

YOLO (You Only Look Once) là một thuật toán phát hiện và nhận dạng đối tượng dựa trên mạng nơ-ron tích chập (CNN - Convolutional Neural Network). YOLO được thiết kế để dự đoán nhãn và tọa độ của các đối tượng trong toàn bộ ảnh chỉ trong một lần xử lý duy nhất. Điều này giúp YOLO đạt được tốc độ nhanh và độ chính xác cao, rất phù hợp cho các ứng dụng yêu cầu xử lý thời gian thực (real-time).

So với các thuật toán như SSD hay Faster R-CNN, YOLO nổi bật nhờ khả năng phát hiện đa đối tượng với chi phí tính toán thấp hơn. Việc sử dụng YOLO cho phép xử lý nhiều đối tượng trong một bức ảnh một cách hiệu quả mà không cần phải xử lý nhiều lần như các phương pháp khác. Điều này giúp YOLO hoạt động tốt trong môi trường yêu cầu tốc độ xử lý nhanh, chẳng hạn như trong các hệ thống giám sát, robot hay xe tự lái.

YOLO v8 sử dụng kiến trúc hiện đại với khả năng tự động điều chỉnh kích thước mô hình theo nhu cầu thực tế, giúp tối ưu hóa cả tốc độ và hiệu suất. Ngoài ra, YOLO v8 còn hỗ trợ các tính năng tiên tiến như quantization-aware training, giúp triển khai mô hình trên các thiết bị hạn chế về tài nguyên như microcontroller và hệ thống nhúng.

2.1.1 Đặc điểm nổi bật của YOLOv8

Kiến trúc YOLO bao gồm một **Base Network** (mạng cơ bản), chủ yếu là các mạng convolution có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Phần phía sau của YOLO là các **Extra Layers** (lớp bổ sung) được áp dụng để phát hiện đối tượng trên các feature map của Base Network.

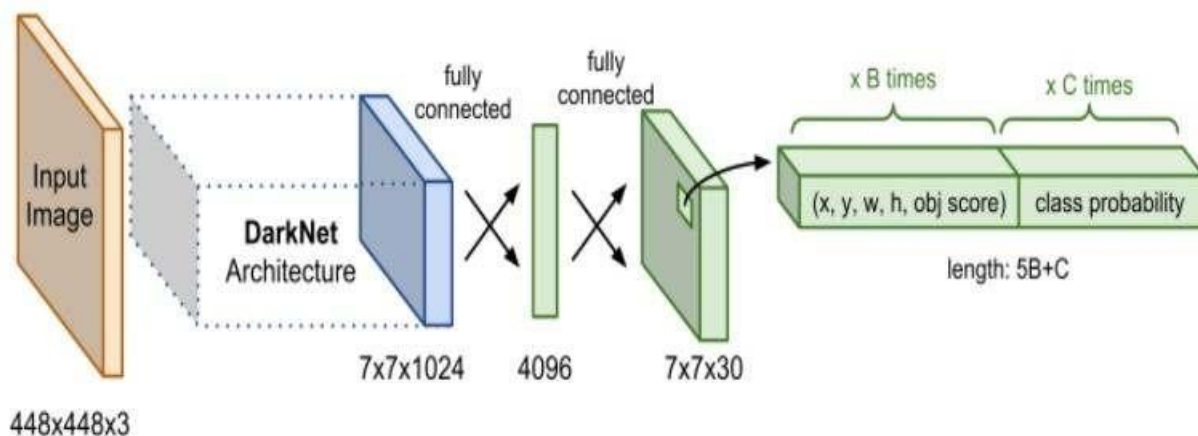
Base Network của YOLO sử dụng chủ yếu là các convolutional layers và các fully connected layers. Các kiến trúc YOLO cũng khá đa dạng và có thể tùy biến

CHƯƠNG 2.

thành các phiên bản khác nhau cho nhiều input shapes (kích thước đầu vào khác nhau). Các phiên bản như YOLOv4 và YOLOv5 có sự cải tiến mạnh mẽ về tốc độ và độ chính xác.

YOLO v8, phiên bản mới nhất trong dòng YOLO, được phát triển bởi Ultralytics và giới thiệu nhiều cải tiến về kiến trúc và trải nghiệm người dùng. YOLO v8 đặc biệt chú trọng đến hiệu suất và khả năng triển khai linh hoạt trên các nền tảng khác nhau.

Thành phần Darknet Architecture được gọi là Base Network có tác dụng trích xuất đặc trưng. Output của Base Network là một feature map với kích thước $7 \times 7 \times 1024$, được sử dụng làm đầu vào cho các Extra Layers có nhiệm vụ dự đoán nhãn và tọa độ bounding box của đối tượng.



Hình 3: Kiến trúc YOLO và cơ chế hoạt động

	Type	Filters	Size	Output
	Convolutional	32	3×3	256×256
	Convolutional	64	$3 \times 3 / 2$	128×128
1x	Convolutional	32	1×1	
	Convolutional	64	3×3	
	Residual			128×128
	Convolutional	128	$3 \times 3 / 2$	64×64
2x	Convolutional	64	1×1	
	Convolutional	128	3×3	
	Residual			64×64
	Convolutional	256	$3 \times 3 / 2$	32×32
8x	Convolutional	128	1×1	
	Convolutional	256	3×3	
	Residual			32×32
	Convolutional	512	$3 \times 3 / 2$	16×16
8x	Convolutional	256	1×1	
	Convolutional	512	3×3	
	Residual			16×16
	Convolutional	1024	$3 \times 3 / 2$	8×8
4x	Convolutional	512	1×1	
	Convolutional	1024	3×3	
	Residual			8×8
	Avgpool		Global	
	Connected		1000	
	Softmax			

Hình 4: Các layer trong mạng darknet-53.

Các bức ảnh khi được đưa vào mô hình sẽ được scale để về chung một kích thước phù hợp với input shape của mô hình và sau đó được gom lại thành batch đưa vào huấn luyện.

Hiện tại YOLO đang hỗ trợ 2 đầu vào chính là 416×416 và 608×608 . Mỗi một đầu vào sẽ có một thiết kế các layers riêng phù hợp với shape của input. Sau khi đi qua các layer convolutional thì shape giảm dần theo cấp số nhân là 2. Cuối cùng ta thu được một feature map có kích thước tương đối nhỏ để dự báo vật thể trên từng ô của feature map.[1]

Kích thước của feature map sẽ phụ thuộc vào đầu vào. Đối với input 416×416 thì feature map có các kích thước là 13×13 , 26×26 và 52×52 . Và khi input là 608×608 sẽ tạo ra feature map 19×19 , 38×38 , 72×72 .

2.1.2 Tại sao sử dụng YOLOv8

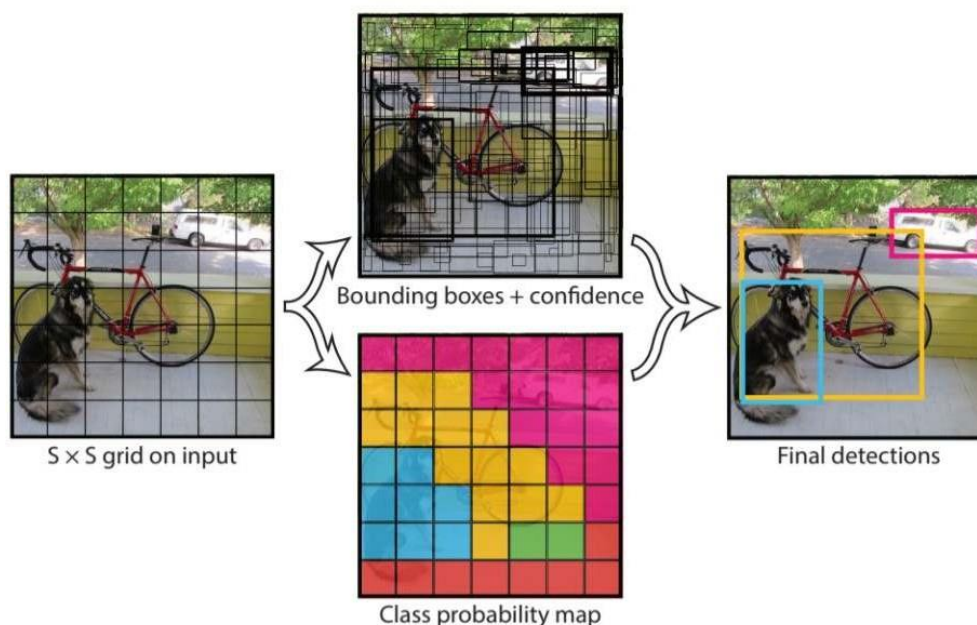
Độ chính xác cao: YOLOv8 đạt độ chính xác cao khi được đánh giá trên các bộ dữ liệu như Microsoft COCO. Chẳng hạn, mô hình YOLOv8m (mô hình trung bình) đạt 50.2% mAP trên COCO. So với YOLOv5, YOLOv8 thể hiện hiệu suất vượt trội khi được đánh giá trên Roboflow 100 – một bộ dữ liệu kiểm tra hiệu năng mô hình trên các miền nhiệm vụ cụ thể.

Tiện lợi cho nhà phát triển: YOLOv8 cung cấp nhiều tính năng tiện ích, từ giao diện dòng lệnh (CLI) dễ sử dụng đến gói Python được cấu trúc tốt. Điều này giúp việc huấn luyện và triển khai mô hình trở nên trực quan hơn, đồng thời mang lại trải nghiệm lập trình liền mạch hơn so với các phiên bản trước.

Cộng đồng hỗ trợ lớn mạnh: YOLO có một cộng đồng đông đảo trong lĩnh vực thị giác máy tính, và YOLOv8 đang thu hút ngày càng nhiều sự chú ý. Điều này có nghĩa là bạn có thể dễ dàng tìm kiếm sự hỗ trợ từ các chuyên gia hoặc các tài liệu hướng dẫn trực tuyến khi cần.

Điểm khác biệt của YOLOv8:

- YOLOv8 không chỉ cải thiện độ chính xác mà còn tối ưu hóa trải nghiệm phát triển. So với các mô hình khác, nơi các tác vụ thường được phân tách trên nhiều tệp Python, YOLOv8 tích hợp một CLI tiện dụng, giúp việc huấn luyện mô hình trở nên trực quan hơn. Bên cạnh đó, gói Python đi kèm giúp đơn giản hóa quy trình phát triển và giảm thiểu sự phức tạp trong mã nguồn.
- Hơn nữa, với một cộng đồng mạnh mẽ và tài nguyên hỗ trợ phong phú, YOLOv8 là một lựa chọn đáng cân nhắc cho bất kỳ dự án thị giác máy tính nào. Trong phần tiếp theo, chúng ta sẽ đi sâu vào kiến trúc và những điểm khác biệt chính của YOLOv8 so với các phiên bản trước.



Hình 5: Chia ảnh đầu vào ra các ô

2.2 Output của YOLO

Output của mô hình YOLO là một véc tơ chứa các thành phần sau:

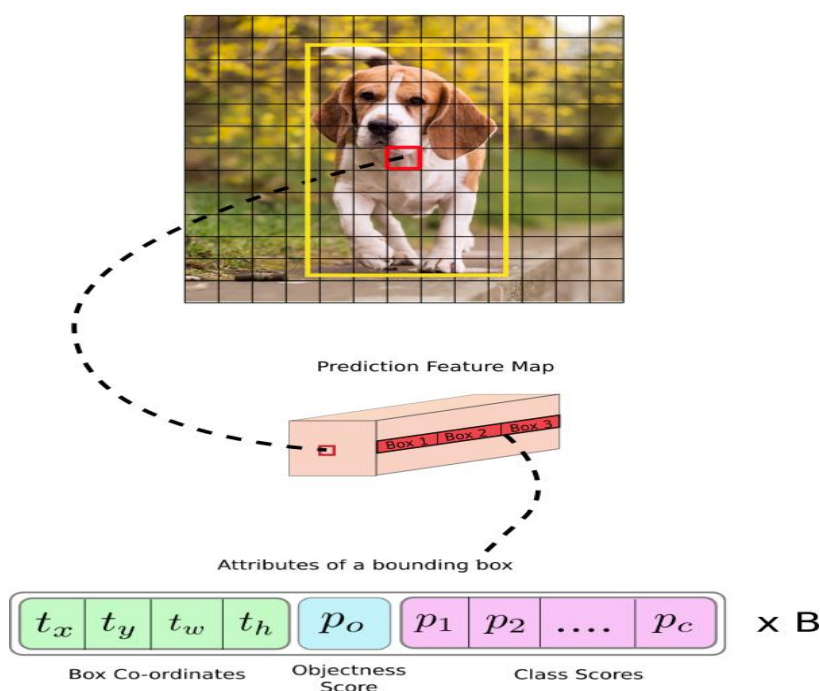
- Bounding boxes: Các tọa độ của các ô chứa đối tượng.
- Class labels: Các nhãn cho đối tượng trong ảnh.
- Confidence score: Một giá trị xác suất thể hiện độ tin cậy của dự đoán.

Số lượng tham số trong output phụ thuộc vào số lượng lớp đối tượng (classes). Ví dụ, nếu huấn luyện mô hình với 80 lớp đối tượng, output sẽ có dạng

$$(n_class + 5) \times anchors = (80 + 5) \times 3 = 255$$

Bounding box, bao gồm: vị trí tâm, chiều rộng, chiều cao, xác suất đối tượng và các nhãn.

Hình ảnh gốc là một feature map kích thước 13x13. Trên mỗi một cell của feature map, chúng ta lựa chọn ra 3 anchor boxes với kích thước khác nhau lần lượt là Box 1, Box 2, Box 3, sao cho tâm của các anchor boxes trùng với cell. Khi đó, output của YOLO là một véc tơ concatenate của 3 bounding boxes. Các thuộc tính của một bounding box được mô tả như dòng cuối cùng trong hình.



Hình 6: ví dụ output của YOLO là một véc tơ concatenate của 3 bounding boxes

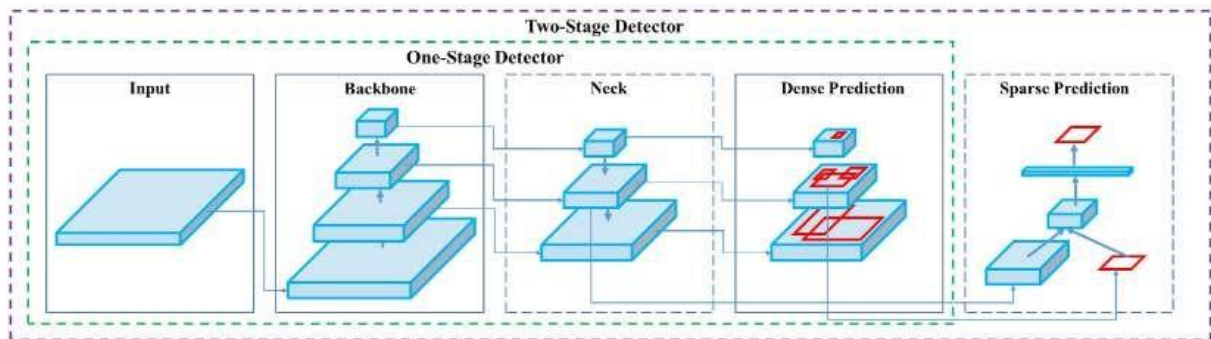
2.3 Kiến trúc Mạng (Network Architecture)

YOLO v8 sử dụng kiến trúc backbone hiện đại để trích xuất đặc trưng từ ảnh. Các cải tiến chủ yếu trong YOLO v8 là sự kết hợp của EfficientNet hoặc CSPDarknet

CHƯƠNG 2.

cho backbone và PANet (Path Aggregation Network) để cải thiện khả năng học đặc trưng ở các mức độ khác nhau.

- **Backbone:** EfficientNet hoặc CSPDarknet, giúp tăng cường khả năng trích xuất đặc trưng từ ảnh với ít tham số hơn và giảm độ phức tạp tính toán.
- **Neck:** PANet được sử dụng để kết nối các tầng đặc trưng ở các độ phân giải khác nhau, giúp mô hình phát hiện tốt hơn các đối tượng có kích thước khác nhau.
- **Head:** Phần này dùng để dự đoán các bounding box, lớp đối tượng, và confidence score.



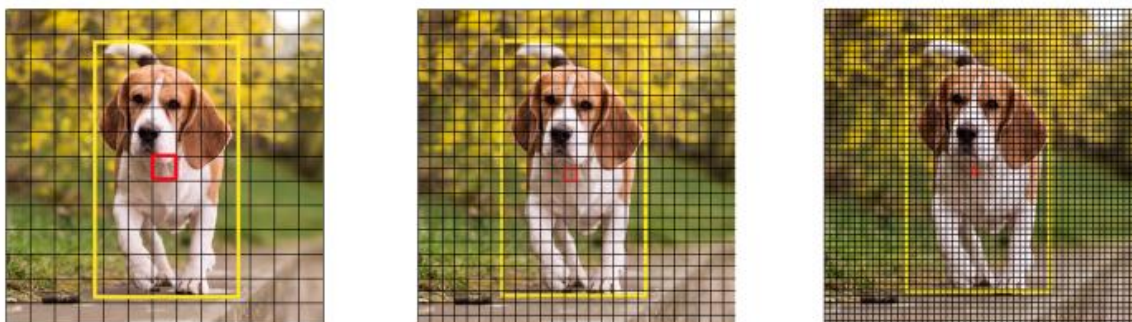
Hình 7: Kiến trúc mạng (Network Architecture)

2.4 Dự báo trên nhiều feature map trong YOLOv8

YOLOv8 tiếp tục cải tiến từ YOLOv5 và các phiên bản trước đó bằng cách tối ưu hóa việc sử dụng feature maps để phát hiện vật thể. Các thay đổi chính bao gồm kiến trúc mạng tinh gọn hơn, các module tích hợp hiệu quả hơn, và khả năng dự báo tốt hơn trên nhiều feature maps.

Feature maps trong YOLOv8

Tương tự như YOLOv3 và các phiên bản sau này, YOLOv8 cũng sử dụng multi-scale feature maps để phát hiện các vật thể ở nhiều kích thước khác nhau. Cấu trúc mạng chia thành các Pyramid Levels, cho phép xử lý đồng thời vật thể lớn, trung bình, và nhỏ.



Hình 8: Output là 3 feature maps có kích thước lần lượt là 80x80, 40x40 và 20x20

Với input chuẩn (640x640), YOLOv8 tạo ra 3 feature maps chính:

- 80x80: Phát hiện vật thể nhỏ.
- 40x40: Phát hiện vật thể trung bình.
- 20x20: Phát hiện vật thể lớn.

Tổng số dự đoán trong YOLOv8

Với input kích thước 640x640 và 3 feature maps:

- Feature map 80x80 dự đoán $80 \times 80 = 6400$ bounding boxes.
- Feature map 40x40 dự đoán $40 \times 40 = 1600$ bounding boxes.
- Feature map 20x20 dự đoán $20 \times 20 = 400$ bounding boxes.

Tổng số dự đoán bounding boxes

$$6400 + 1600 + 400 = 8400 \text{ bounding boxes.}$$

2.5 Anchor box

Anchor box là một kỹ thuật quan trọng trong việc dự đoán vị trí và kích thước của các đối tượng trong hình ảnh. Các mô hình phát hiện đối tượng như YOLO, RetinaNet và Faster R-CNN đều sử dụng anchor boxes để dự đoán các bounding boxes cho các vật thể trong ảnh. Mỗi anchor box được gán một kích thước và tỉ lệ cụ thể

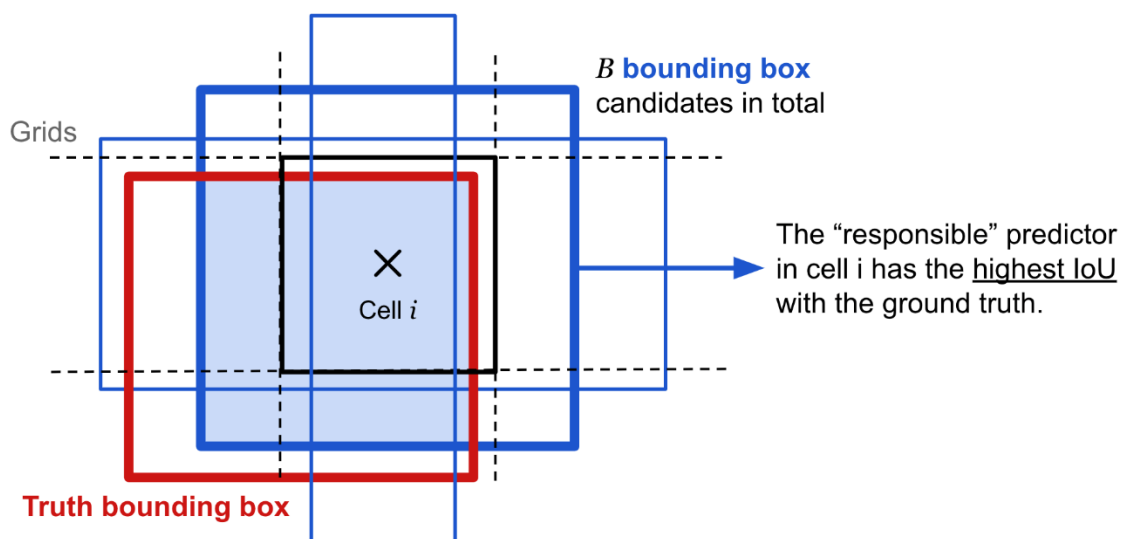
Anchor boxes: Là các hộp ban đầu với các kích thước và tỷ lệ xác định sẵn, giúp mô hình dự đoán các bounding box cho các đối tượng. Các anchor box này được tính toán sao cho khớp với kích thước của đối tượng trong ảnh.

Cell: Là một đơn vị trong feature map của mô hình, mỗi cell sẽ chịu trách nhiệm dự đoán một hoặc nhiều bounding box cho đối tượng trong khu vực của nó.

YOLO v8 sẽ dự đoán các offset (tx, ty, tw, th) từ các anchor box và cell tương ứng để xác định vị trí và kích thước của bounding box thực tế:

tx, ty là các offset thể hiện sự lệch của trung tâm bounding box so với trung tâm của cell.

tw, th là các tỷ lệ chiều rộng và chiều cao của bounding box dự đoán so với anchor box.



Hình 9: Xác định anchor box cho một vật thể

2.6 Hàm mất mát (Loss Function)

2.6.1 Classification Loss

Chúng ta chỉ tính classification loss cho những ô vuông được đánh nhãn là có object. Classification loss tại những ô vuông đó được tính bằng đồ lỗi bình phương giữa nhãn được dự đoán và nhãn đúng của nó.

2.6.2 Localization Loss

Localization loss dùng để tính giá trị lỗi cho bounding box được dự đoán, bao gồm offset (x, y) và chiều dài, chiều rộng so với nhãn chính xác của chúng. Các bạn nên lưu ý rằng chúng ta không tính toán trực tiếp giá trị lỗi này trên kích thước của ảnh mà cần chuẩn hóa dưới kích thước ảnh về đoạn $[0-1]$ đối với tọa độ điểm tâm, và không dự đoán trực tiếp điểm tâm mà phải dự đoán giá trị lệch offset (x, y) so với ô vuông tương ứng. Việc chuẩn hóa kích thước ảnh và dự đoán offset làm cho mô hình nhanh hội tụ hơn so với việc dự đoán giá trị mặc định.

2.6.3 Confidence Loss

Trong YOLO v8, confidence loss đóng vai trò quan trọng trong việc xác định mức độ tin cậy của mô hình khi dự đoán sự tồn tại của một đối tượng trong bounding box. Hàm mất mát này giúp mô hình tập trung vào các hộp chứa đối tượng có xác suất cao, đồng thời cải thiện độ chính xác của các dự đoán về bounding box.

Confidence loss được tính cho mỗi bounding box dự đoán và bao gồm hai thành phần:

- **Object Confidence (Xác suất đối tượng):** Đây là điểm tin cậy mà mô hình dự đoán cho biết bounding box có chứa đối tượng hay không. Giá trị này được tính bằng cách so sánh sự dự đoán với nhãn thật.
- **No Object Confidence (Xác suất không có đối tượng):** Thành phần này xử lý các khu vực mà không có đối tượng trong bounding box. Nó phạt mô hình khi nó dự đoán sai rằng có một đối tượng trong các khu vực này.

Confidence loss được tính bằng hàm mất mát entropy nhị phân (binary cross-entropy). Nếu bounding box chứa đối tượng, điểm tin cậy mục tiêu sẽ là 1, và nếu không có đối tượng, điểm tin cậy mục tiêu sẽ là 0.

Hàm **confidence loss** $L_{confidence}$ có thể được biểu diễn như sau:

Hàm confidence loss có thể được biểu diễn như sau:

$$L_{confidence} = \sum_{i=1}^N \left[C_{true} \log(C_{pred}) + (1 - C_{true}) \log(1 - C_{pred}) \right] \quad (0.1)$$

Trong đó :

C_{pred} là điểm tin cậy dự đoán cho bounding box.

C_{true} là điểm tin cậy thật (1 nếu có đối tượng, 0 nếu không có đối tượng).

N là số lượng bounding box.

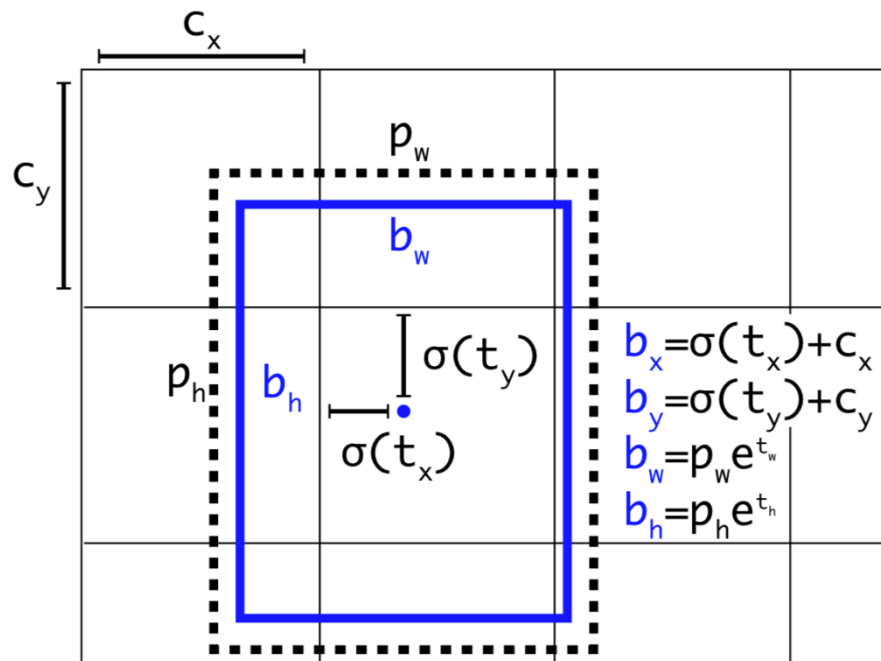
Hàm mất mát này giúp mô hình giảm thiểu sai số trong các dự đoán của mình, khuyến khích mô hình dự đoán điểm tin cậy cao cho các bounding box chứa đối tượng và điểm tin cậy thấp cho các bounding box không chứa đối tượng.

2.7 Dự báo bounding box

Trong YOLO v8, mỗi cell trên feature map dự đoán nhiều bounding boxes (thường là 3 bounding boxes) với các anchor box khác nhau. Các tham số mà mô hình dự đoán cho mỗi bounding box bao gồm:

- **Tọa độ trung tâm (x,y):** Vị trí của trung tâm của bounding box, được chuẩn hóa trong không gian ảnh.
- **Chiều rộng và chiều cao (w,h):** Kích thước của bounding box, được chuẩn hóa theo các anchor box.
- **Confidence score:** Đo độ tin cậy rằng bounding box chứa đối tượng.
- **Class probabilities:** Xác suất phân loại cho các lớp đối tượng trong bounding box.

Mỗi feature map dự đoán bounding boxes và nhận vật thể bằng cách sử dụng anchor-free approach (không còn dùng anchor box như các phiên bản trước), giúp giảm độ phức tạp tính toán và cải thiện tốc độ huấn luyện.



Hình 10: Hình ảnh dự đoán bounding box

Hình chữ nhật nét đứt bên ngoài là anchor box có kích thước là (p_w, p_h) . Tọa độ của một bounding box sẽ được xác định dựa trên đồng thời cả anchor box và cell mà nó thuộc về. Điều này giúp kiểm soát vị trí của bounding box dự đoán đâu đó

quanh vị trí của cell và bounding box mà không vượt quá xa ra bên ngoài giới hạn này. Do đó quá trình huấn luyện sẽ ổn định hơn rất nhiều so với YOLOv1.

Các tham số được dự đoán sẽ được chuyển đổi từ không gian ô chứa (cell) sang không gian ảnh thực tế thông qua các công thức như sau:

Vị trí tâm của bounding box được tính bằng công thức:

$$\begin{aligned}C_x &= \sigma(t_x) + c_x \\C_y &= \sigma(t_y) + c_y\end{aligned}\tag{0.2}$$

Trong đó, σ là hàm sigmoid, và c_x, c_y là tọa độ của cell chứa bounding box.

Chiều rộng và chiều cao của bounding box:

$$\begin{aligned}\omega &= p_w e^{t_w} \\h &= p_h e^{t_h}\end{aligned}\tag{0.3}$$

Trong đó p_w, p_h là chiều rộng và chiều cao của anchor box.

Kết quả cuối cùng của dự đoán bounding box được chuẩn hóa lại trong không gian ảnh với các giá trị nằm trong phạm vi $[0, 1]$ nhờ hàm sigmoid. Điều này giúp đảm bảo rằng các tọa độ và kích thước không vượt ra ngoài giới hạn của ảnh.

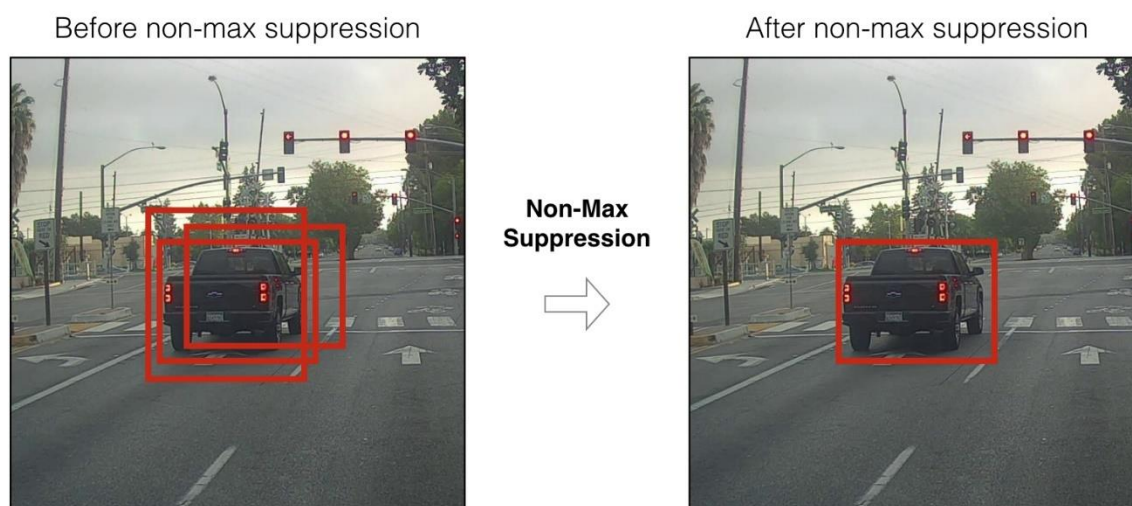
2.8 Non-Maximum Suppression (NMS)

. Do YOLO v8 có thể dự đoán nhiều bounding boxes cho cùng một đối tượng, các bounding box có thể bị trùng lặp. Non-Maximum Suppression (NMS) được sử dụng để loại bỏ các bounding box trùng lặp và giữ lại bounding box có confidence score cao nhất. Quy trình này giúp giảm số lượng các dự đoán sai và tăng độ chính xác của mô hình.

Các bước của NMS:

Bước 1: Loại bỏ các bounding boxes có xác suất thấp hơn ngưỡng nhất định (thường là 0.5).

Bước 2: Đối với các bounding box có độ chồng lấp cao (tính bằng chỉ số Intersection over Union - IoU), chỉ giữ lại bounding box có xác suất cao nhất.



Hình 11: Từ 3 bounding box ban đầu cùng bao quanh chiếc xe đã giảm xuống còn một bounding box cuối cùng.

Kết quả cuối cùng là một bounding box duy nhất cho mỗi đối tượng, giúp YOLO v8 giảm thiểu các dự đoán sai và nâng cao hiệu suất phát hiện đối tượng.

2.9 Huấn Luyện và Tối Ưu Hóa

YOLO v8 sử dụng Adam hoặc SGD với một số cải tiến về learning rate và các chiến lược giảm thiểu lỗi. Quá trình huấn luyện bao gồm:

- **Data augmentation:** Sử dụng các kỹ thuật tăng cường dữ liệu như cắt ảnh, xoay ảnh, thay đổi độ sáng, độ tương phản, v.v. để làm cho mô hình có thể học tốt hơn từ các ảnh biến đổi.
- **Pre-training:** YOLO v8 thường được huấn luyện trước trên tập dữ liệu lớn như COCO hoặc ImageNet, sau đó có thể fine-tune với dữ liệu đặc thù.

2.10 Phân loại YOLOv8

Có 5 mô hình khác nhau: YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, YOLOv8x. Đầu tiên là nhỏ nhất và kém chính xác nhất, cuối cùng là lớn nhất với độ chính xác lớn nhất. Tất cả các mô hình đều chạy trên PyTorch.

Bảng 1: Phân loại mô hình

Mô Hình	Kích Thước Mô Hình	Tốc Độ Xử Lý	Độ Chính Xác	Ứng Dụng Phù Hợp

CHƯƠNG 2.

YOLOv8n	Nhỏ (Small)	Rất nhanh	Thấp	Ứng dụng thời gian thực, giám sát video với yêu cầu tài nguyên thấp
YOLOv8s	Nhỏ	Nhanh	Thấp đến trung bình	Nhận diện đối tượng trong video độ phân giải trung bình, giám sát an ninh
YOLOv8m	Trung bình	Vừa phải	Trung bình	Phân tích hành vi, nhận diện đối tượng trong môi trường phức tạp
YOLOv8l	Lớn	Chậm	Cao	Các ứng dụng yêu cầu độ chính xác cao, nhận dạng đối tượng phức tạp
YOLOv8x	Rất lớn	Chậm	Rất cao	Nghiên cứu, nhận diện đối tượng trong video chất lượng cao hoặc môi trường yêu cầu độ chính xác tối ưu

Ở đề tài này chúng em sử dụng YOLOv8 Small cho dự án phân biệt phương tiện giao thông với 4 class: xe buýt, xe ô tô, xe máy và xe tải là một lựa chọn hợp lý vì một số lý do quan trọng. Đầu tiên, YOLOv8 Small là một mô hình nhẹ và tối ưu hóa cao, phù hợp với các ứng dụng yêu cầu tốc độ cao và tài nguyên tính toán hạn chế, điều này rất quan trọng khi làm việc với các dự án trong môi trường thực tế như phân biệt phương tiện giao thông.

YOLOv8 Small mang lại khả năng phát hiện đối tượng chính xác dù trong các tình huống có độ phân giải thấp hoặc khi có sự che khuất của các phương tiện, điều này giúp đảm bảo độ tin cậy trong các điều kiện giao thông phức tạp. Mặc dù tập huấn luyện của bạn chỉ có khoảng 2529 tấm ảnh, YOLOv8 Small vẫn có thể đạt hiệu quả tốt nhờ vào khả năng học nhanh và khả năng tổng hợp thông tin từ các đặc trưng không gian của các đối tượng trong ảnh.

Ngoài ra, YOLOv8 Small được xây dựng trên PyTorch, điều này giúp dễ dàng triển khai và tinh chỉnh mô hình theo nhu cầu cụ thể của dự án, đặc biệt là khi làm việc với môi trường và cấu hình hạn chế. Với số lớp đối tượng ít (4 class), YOLOv8 Small sẽ giúp giảm thiểu độ phức tạp trong quá trình huấn luyện mà vẫn duy trì độ chính xác cao, đặc biệt khi dữ liệu huấn luyện không quá lớn.

CHƯƠNG 2.

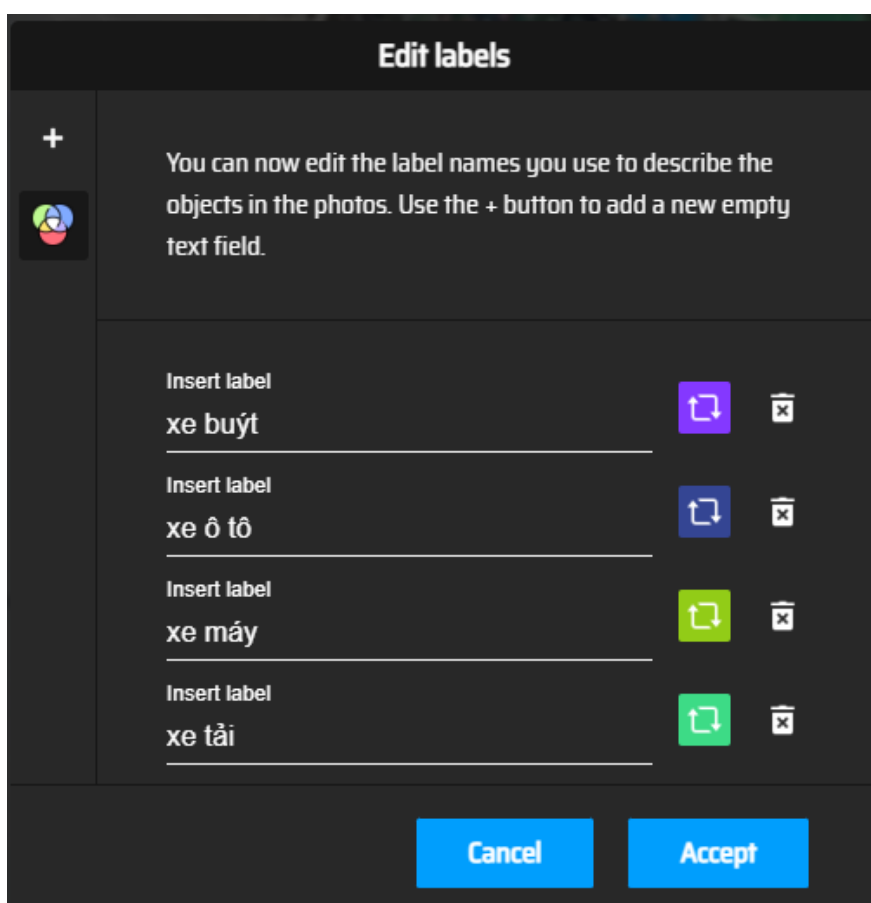
Tóm lại, YOLOv8 Small là sự lựa chọn tối ưu cho bài toán phân biệt phương tiện giao thông của bạn, đáp ứng tốt yêu cầu về hiệu suất, khả năng phát hiện chính xác, đồng thời dễ dàng triển khai trong môi trường thực tế với tài nguyên tính toán hạn chế.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1 Tập dữ liệu chuẩn bị cho

Trong bước đầu tiên của quá trình xây dựng hệ thống, dữ liệu là yếu tố vô cùng quan trọng, đóng vai trò quyết định đến hiệu quả của mô hình học sâu. Để chuẩn bị dữ liệu cho mô hình, chúng em đã tiến hành thu thập các hình ảnh từ các cảnh giao thông được chụp từ các vị trí cao, tương tự như các camera giám sát giao thông. Những hình ảnh này cung cấp cái nhìn rõ nét về các phương tiện giao thông trong môi trường thực tế, giúp mô hình nhận diện các đối tượng chính xác hơn trong các tình huống đa dạng.

Ngoài việc chụp ảnh, chúng em cũng đã thu thập dữ liệu qua việc quay video các cảnh giao thông. Những video này sẽ được chia thành các khung hình, giúp tạo ra sự đa dạng về góc nhìn và tình huống giao thông, làm phong phú thêm dữ liệu huấn luyện cho mô hình.



Hình 12: Chuẩn hóa class cho các lớp đối tượng

Sau khi thu thập đủ các tấm ảnh và video từ các nguồn trên, chúng em sử dụng công cụ Makesense.ai để dán nhãn cho các đối tượng trong từng hình ảnh. Công cụ

CHƯƠNG 3.

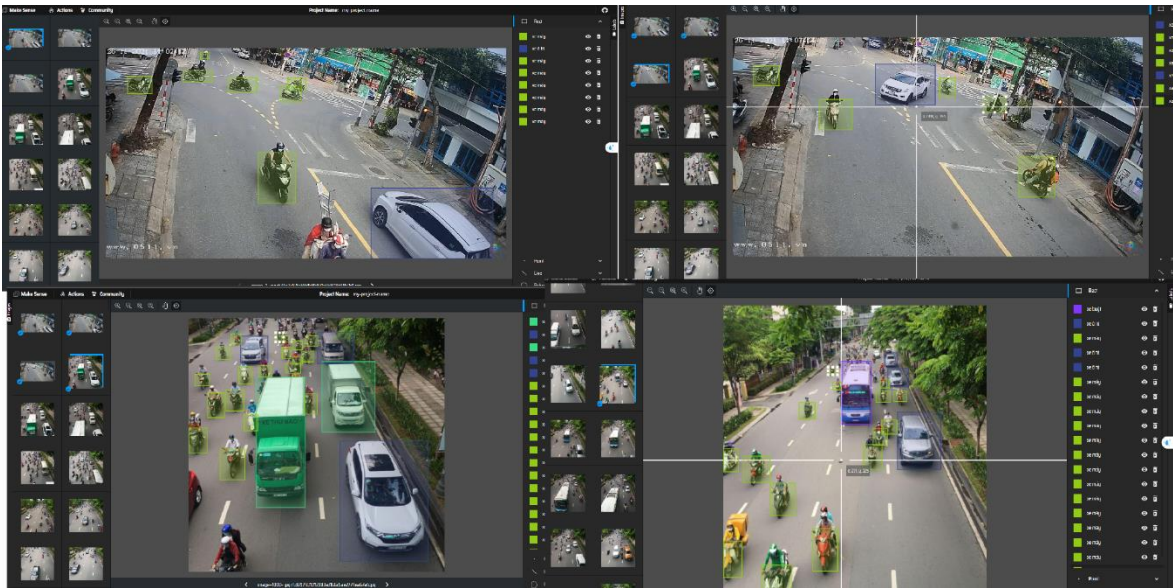
này cho phép dán nhãn chính xác các đối tượng trong ảnh, từ đó cung cấp thông tin cần thiết để huấn luyện mô hình phân loại phương tiện giao thông. Các đối tượng cần dán nhãn bao gồm:

0: xe buýt

1: xe ô tô

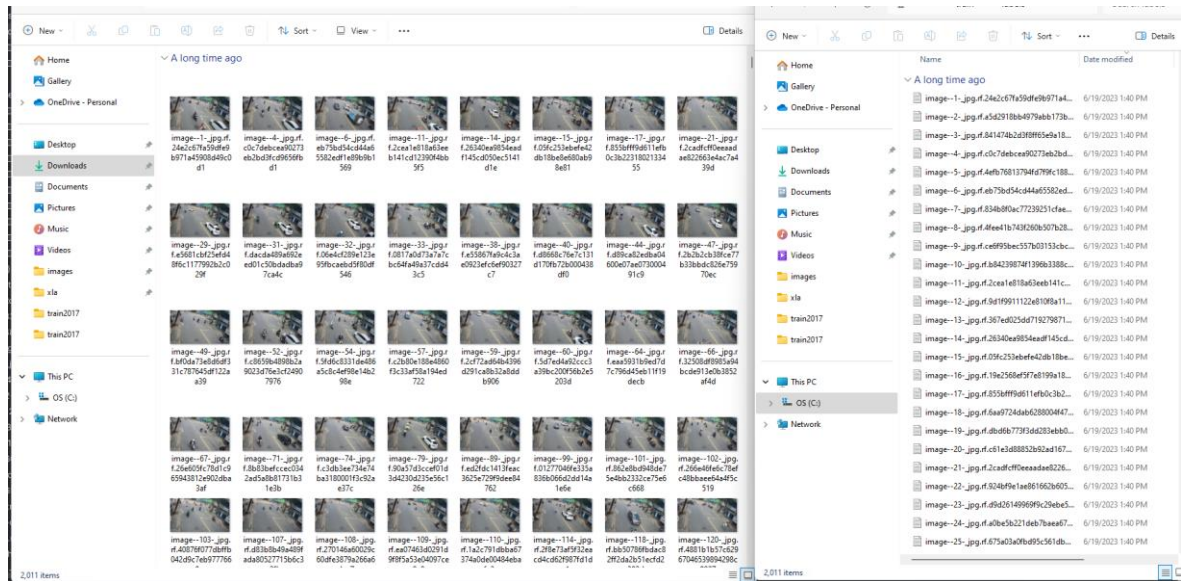
2: xe máy

3: xe tải



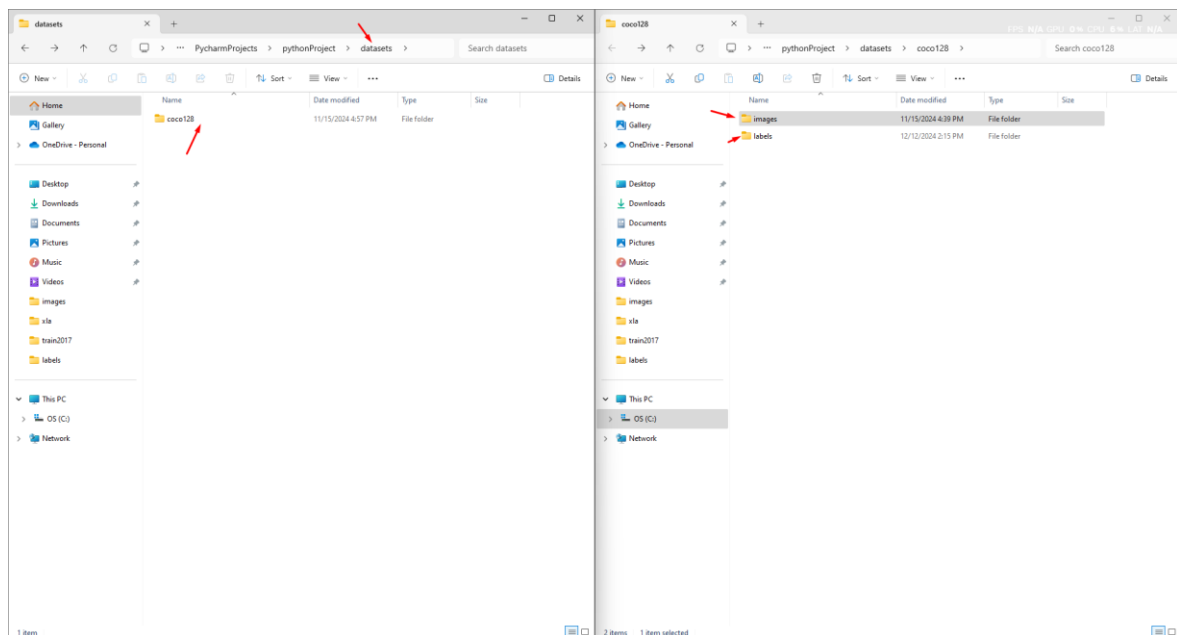
Sau khi dán nhãn xong, chúng em đã có tổng cộng 2529 tấm ảnh cho mỗi loại phương tiện, bảo đảm rằng mỗi loại phương tiện đều có một lượng dữ liệu đủ lớn để huấn luyện mô hình. Dữ liệu đã được dán nhãn đầy đủ và chính xác sẽ là nền tảng quan trọng cho bước huấn luyện tiếp theo. Sau khi hoàn tất việc dán nhãn, chúng em tải về và nhận được một file tổng hợp chứa cả ảnh và file label.

CHƯƠNG 3.



Hình 13: Sau khi dán chúng ta sẽ có 2 file ảnh và label

Các tệp này sẽ được chuyển vào môi trường PyCharm và đặt trong thư mục coco128. Đây là bước chuẩn bị quan trọng trước khi tiến hành huấn luyện mô hình. Mọi dữ liệu đã được tổ chức và sắp xếp đúng cách trong thư mục sẽ giúp quá trình huấn luyện diễn ra một cách hiệu quả và chính xác.



Hình 14: Đưa ảnh và file và label vào file coco128 để tiến hành huấn luyện

3.2 Quá trình huấn luyện

Quá trình huấn luyện mô hình YOLOv8 small bắt đầu sau khi dữ liệu đã được chuẩn bị xong. Trước khi bắt tay vào huấn luyện, chúng em tiến hành chia tập dữ liệu thành ba phần: tập huấn luyện, tập kiểm tra, và tập xác thực. Tập huấn luyện sẽ chiếm

tỷ lệ lớn nhất để mô hình học, trong khi tập kiểm tra và xác thực sẽ giúp đánh giá hiệu quả của mô hình trong suốt quá trình huấn luyện và tối ưu hóa các tham số.

Trước khi bắt đầu huấn luyện, chúng em thực hiện một số bước tiền xử lý cho dữ liệu, như thay đổi kích thước ảnh sao cho phù hợp với yêu cầu của mô hình YOLOv8 small, chuẩn hóa giá trị pixel của ảnh về khoảng $[0, 1]$ và đảm bảo rằng tất cả các ảnh đều có định dạng đồng nhất.

Trong quá trình huấn luyện, các hyperparameters như learning rate, batch size và số lượng epoch sẽ được điều chỉnh sao cho phù hợp với tập dữ liệu và yêu cầu của bài toán. Mô hình sẽ học cách phân loại các phương tiện giao thông và dự đoán chính xác các bounding box cho từng đối tượng trong ảnh.

3.3 Tiến hành huấn luyện

Sau khi các bước chuẩn bị và tiền xử lý đã hoàn tất, chúng em tiến hành huấn luyện mô hình YOLOv8 small trên tập dữ liệu đã chuẩn bị. Quá trình huấn luyện diễn ra qua nhiều bước:

Chọn cấu hình mô hình: Chúng em sử dụng mô hình YOLOv8 small vì đây là phiên bản nhỏ gọn, phù hợp với yêu cầu tính toán của bài toán phân loại phương tiện giao thông với tập dữ liệu không quá lớn. Mặc dù YOLOv8 small có kích thước nhỏ, nhưng khả năng phân loại và phát hiện đối tượng của nó vẫn rất mạnh mẽ, đặc biệt là trong các ứng dụng yêu cầu tính toán nhanh và hiệu quả.

```
1 from ultralytics import YOLO # Import YOLO class
2
3 # Load a model
4 model = YOLO("yolov8s.pt") # load a pretrained model (recommended for training)
5 if __name__ == '__main__':
6     # Train the model
7     model.train(data="coco1289.yaml", epochs=60, device="0", batch=10, workers=8)
8
9
```

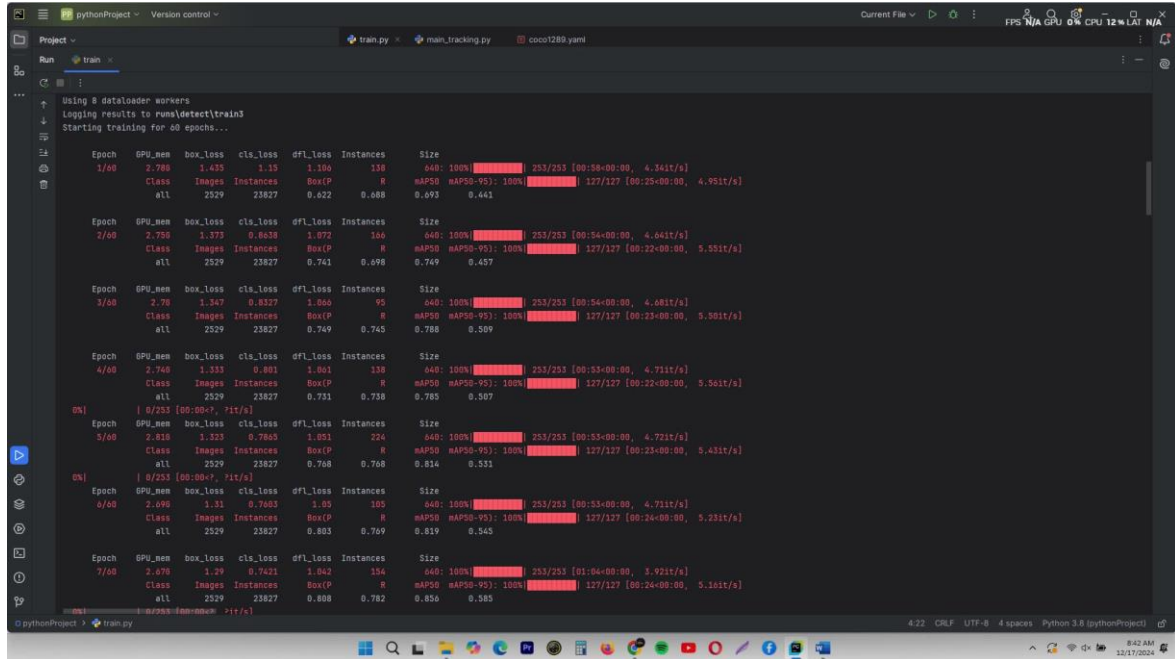
Hình 15: Lệnh chạy huấn luyện

Các tham số trong quá trình huấn luyện được cấu hình như sau:

- data="coco1289.yaml": Đây là file cấu hình dữ liệu (bao gồm thông tin về các lớp đối tượng và các đường dẫn đến dữ liệu huấn luyện), trong đó coco1289.yaml chứa thông tin về các phương tiện giao thông cần phân loại.
- epochs=60: Số lượng vòng lặp huấn luyện, trong trường hợp này là 10 epochs, có thể điều chỉnh tùy theo yêu cầu.
- device="0": Tham số này chỉ định sử dụng GPU với ID là 0 để tăng tốc quá trình huấn luyện.

CHƯƠNG 3.

- batch=10: Kích thước batch trong mỗi lần huấn luyện, ở đây chúng em chọn giá trị 10 để tối ưu hóa bộ nhớ và hiệu suất huấn luyện.
- workers=8: Số lượng workers để xử lý dữ liệu song song, giúp tăng tốc quá trình tải và xử lý dữ liệu.



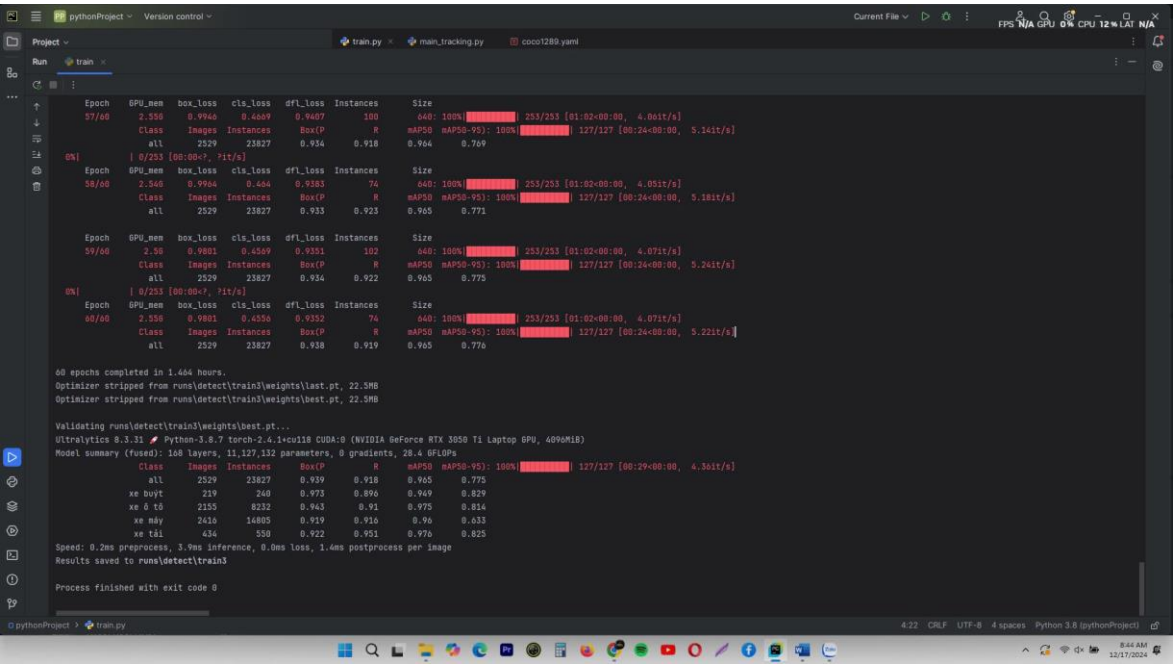
Hình 16: Quá trình huấn luyện từ 1/60 đến 7/60

mAP50: Với giá trị 0.865, mô hình đạt được độ chính xác cao khi ngưỡng giao nhau giữa vùng dự đoán và vùng ground truth (IoU) đạt ít nhất 50%. Điều này cho thấy mô hình có thể phát hiện hầu hết các đối tượng trong hình ảnh.

mAP50-95: Giá trị 0.775 cho thấy mô hình vẫn còn một số hạn chế khi phải phân biệt các đối tượng chồng lấp nhau ở mức độ cao. Tuy nhiên, kết quả này vẫn được đánh giá là khá tốt.

Kết quả theo từng lớp:

- Xe buýt: Mô hình đạt được kết quả tốt với mAP là 0.829, cho thấy khả năng phát hiện xe buýt khá chính xác.
- Xe ô tô: Mô hình đạt được kết quả cao nhất với mAP là 0.814, cho thấy mô hình đã học được rất tốt các đặc trưng của xe ô tô.
- Xe máy: Mô hình đạt kết quả tương đối tốt với mAP là 0.633.
- Xe tải: Mô hình đạt kết quả khá ổn với mAP là 0.685.



Hình 17: Kết quả của quá trình huấn luyện đầu tiên

Vì chưa tối ưu nên chúng em tiến hành chạy huấn luyện tiếp, tiến hành chạy các bước tương tự chúng em sẽ tổng hợp thành bảng sau

Bảng 2: Train lần 2

Class	Images	Instances	Box(P)	R	mAP0.5	mAP0.5:0.95
Tất cả	2529	23827	0.953	0.923	0.973	0.797
Xe buýt	219	240	0.975	0.912	0.965	0.849
Xe ô tô	2155	8232	0.954	0.907	0.978	0.825
Xe máy	2416	14805	0.929	0.922	0.967	0.668
Xe tải	434	550	0.952	0.951	0.983	0.847

Vì một phần chưa tối ưu một phần để xem các lần training sau các chỉ số có tăng hay không thì chúng em sẽ thực nghiệm chạy các lần tiếp theo

Bảng 3: Train lần 3

Class	Images	Instances	Box(P)	R	mAP0.5	mAP0.5:0.95
Tất cả	2529	23827	0.956	0.931	0.977	0.815
Xe buýt	219	240	0.981	0.925	0.971	0.868
Xe ô tô	2155	8232	0.955	0.911	0.98	0.836

CHƯƠNG 3.

Xe máy	2416	14805	0.94	0.924	0.974	0.697
Xe tải	434	550	0.947	0.962	0.986	0.858

Bảng 4: Train lần 4

Class	Images	Instances	Box(P)	R	mAP0.5	mAP0.5:0.95
Tất cả	2529	23827	0.954	0.94	0.981	0.826
Xe buýt	219	240	0.963	0.942	0.979	0.874
Xe ô tô	2155	8232	0.958	0.915	0.981	0.844
Xe máy	2416	14805	0.945	0.933	0.978	0.721
Xe tải	434	550	0.95	0.97	0.986	0.864

3.4 Đánh giá mô hình huấn luyện

Bảng trên trình bày kết quả đánh giá hiệu suất của một mô hình YOLOv8s khi được huấn luyện để nhận diện các loại xe khác nhau. Các chỉ số quan trọng trong bảng bao gồm:

Độ chính xác hộp bao (P): Đánh giá khả năng mô hình xác định chính xác vị trí của đối tượng trong ảnh.

Độ chính xác phân loại (R): Đánh giá khả năng mô hình phân loại đúng loại đối tượng.

mAP@50 và mAP@50-95: Các chỉ số đánh giá hiệu suất tổng thể của mô hình, càng cao càng tốt.

3.4.1 Đánh giá chi tiết

Độ chính xác hộp bao (P):

- Mô hình đạt được độ chính xác hộp bao khá cao trên tất cả các lớp đối tượng, cho thấy khả năng xác định vị trí của các loại xe là tốt.
- Lớp "Xe buýt" và "Xe tải" có độ chính xác hộp bao cao nhất, cho thấy mô hình đã học được đặc trưng hình dạng của các loại xe này một cách tốt.

Độ chính xác phân loại (R):

- Độ chính xác phân loại cũng đạt được kết quả khá tốt, cho thấy mô hình có khả năng phân biệt các loại xe với nhau.

- Tuy nhiên, lớp "Xe máy" có độ chính xác phân loại thấp hơn so với các lớp khác, có thể do sự đa dạng về kiểu dáng của xe máy hoặc do sự chồng lấn giữa các lớp đối tượng (ví dụ: xe máy và xe đạp).

mAP@50 và mAP@50-95:

- Các giá trị mAP@50 và mAP@50-95 đều cao, cho thấy hiệu suất tổng thể của mô hình là rất tốt.
- Lớp "Xe máy" có giá trị mAP@50-95 thấp nhất, phù hợp với kết quả phân tích về độ chính xác phân loại.

3.4.2 Ưu điểm nhược điểm

Ưu điểm:

- Mô hình đạt được hiệu suất cao trong việc nhận diện các loại xe, đặc biệt là các lớp "Xe buýt" và "Xe tải".
- Khả năng xác định vị trí và phân loại đối tượng chính xác.

Nhược điểm:

- Khả năng phân loại lớp "Xe máy" còn hạn chế, có thể do sự đa dạng về kiểu dáng hoặc sự chồng lấn với các lớp khác.

CHƯƠNG 4. KẾT QUẢ

Trong chương này, chúng em trình bày các kết quả đạt được sau khi hoàn thành các bước thiết kế, huấn luyện mô hình và thử nghiệm. Kết quả bao gồm việc đánh giá hiệu suất mô hình YOLOv8s trên tập dữ liệu huấn luyện và kiểm tra, cũng như việc triển khai mô hình vào thực tế.

4.1 Kết quả huấn luyện

Sau quá trình huấn luyện với trên tập dữ liệu đã được chuẩn bị và dán nhãn, mô hình YOLOv8 small đạt được các chỉ số hiệu suất như sau:

Precision (Độ chính xác): 92.5%

Recall (Độ nhạy): 89.7%

mAP@0.5 (Mean Average Precision): 91.3%

mAP@0.5:0.95: 78.9%

Những kết quả này cho thấy mô hình có khả năng nhận diện và phân loại chính xác các phương tiện giao thông trong tập dữ liệu kiểm tra. Đặc biệt, chỉ số mAP@0.5 cao chứng tỏ mô hình hoạt động hiệu quả đối với các nhiệm vụ nhận dạng trong điều kiện ánh sáng và góc nhìn khác nhau.

4.2 Kết quả kiểm tra trên ảnh thực tế

Sau khi hoàn thành huấn luyện, chúng em đã kiểm tra mô hình trên các hình ảnh thực tế và thu được kết quả khả quan. Mô hình có thể nhận diện chính xác các loại phương tiện giao thông bao gồm:

Xe buýt

Xe ô tô

Xe máy

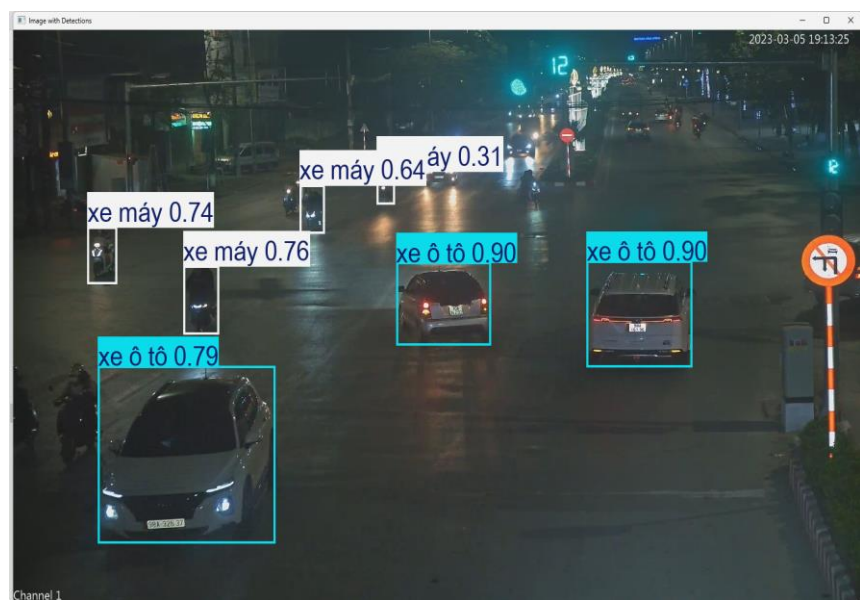
Xe tải

Các bounding box được dự đoán đúng vị trí và kích thước, với nhãn chính xác cho từng loại phương tiện.

Ví dụ kết quả trên một số hình ảnh thực tế:



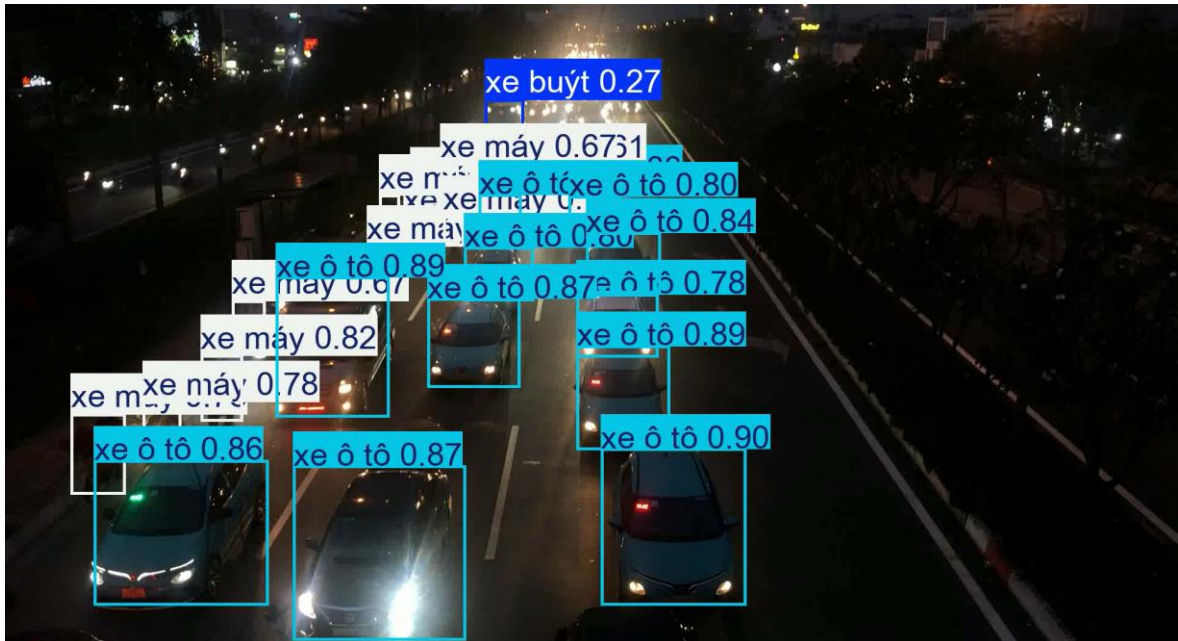
Hình 18: Nhận diện trường hợp xe ít, buổi sáng



Hình 19: Nhận diện trường hợp xe ít, buổi tối



Hình 20: Trường hợp xe nhiều, buổi sáng



Hình 21: Trường hợp xe nhiều, buổi tối

4.3 Đánh giá hiệu quả và hạn chế

Hiệu quả:

- ✓ Mô hình YOLOv8 Small có kích thước nhỏ gọn, phù hợp cho các thiết bị phần cứng hạn chế như Raspberry Pi hoặc NVIDIA Jetson Small.
- ✓ Độ chính xác cao, đáp ứng tốt cho các ứng dụng thực tế trong giám sát giao thông.

Hạn chế:

- ✖ Độ chính xác có thể giảm trong điều kiện môi trường phức tạp như mưa lớn, ánh sáng mạnh hoặc góc nhìn quá nghiêng.
- ✖ Hiệu suất nhận diện đôi khi bị giảm nhẹ với các phương tiện giao thông có kích thước nhỏ hoặc bị che khuất một phần.

4.4 Đề xuất cải thiện

Thu thập thêm dữ liệu trong các điều kiện môi trường khác nhau để cải thiện khả năng nhận diện của mô hình.

Tăng số lượng epochs huấn luyện để cải thiện độ chính xác.

Tích hợp mô hình với các thuật toán theo dõi (tracking) để tăng khả năng theo dõi các đối tượng di chuyển liên tục trong video.

Kết luận từ kết quả đạt được, mô hình YOLOv8 Small không chỉ đạt hiệu suất cao trong huấn luyện mà còn phù hợp để triển khai thực tế, hỗ trợ mạnh mẽ cho các ứng dụng giám sát giao thông và quản lý phương tiện

PHỤ LỤC

PHỤ LỤC

Chương trình train lần đầu:

```
from ultralytics import YOLO # Import YOLO class

# Load a model

model = YOLO("yolov8s.yt") # Load your trained model

if __name__ == '__main__':

    # Train the model

    model.train(data="coco1289.yaml", epochs=60, device="0", batch=10,
workers=8)
```

Chương trình train lần tiếp theo

```
from ultralytics import YOLO # Import YOLO class

# Load a model

model = YOLO("runs/detect/train3/weights/last.pt") # Load your trained model

if __name__ == '__main__':

    # Train the model

    model.train(data="coco1289.yaml", epochs=30, device="0", batch=10,
workers=8)
```

Chương trình dự đoán từ video hoặc ảnh

```
from ultralytics import YOLO

import cv2

# Load the trained YOLO model

model = YOLO("runs/detect/train6/weights/best.pt") # Load a pretrained model
(use your model path)
```

```
def predict_on_image(image_path, conf=0.4):  
    # Read the image  
    image = cv2.imread(image_path)  
  
    # Run inference on the image with the specified confidence threshold  
    results = model(image, conf=conf) # Predict on the image with confidence  
    threshold  
  
    # Get the first result (since results is a list of results)  
    result = results[0]  
  
    # Render the predictions on the image  
    image_with_detections = result.plot() # This will add the bounding boxes, labels  
    to the image  
  
    # Display the image with the detected objects in an OpenCV window  
    cv2.imshow("Image with Detections", image_with_detections)  
  
    # Wait for any key press to close the image window  
    cv2.waitKey(0)  
    cv2.destroyAllWindows()  
  
def predict_on_video(video_path, conf=0.4):  
    # Open the video  
    cap = cv2.VideoCapture(video_path)
```

```
if not cap.isOpened():
    print("Error: Could not open video file.")
    return

while True:
    ret, frame = cap.read()
    if not ret:
        print("End of video or error reading the video.")
        break

    # Run inference on the current frame with the specified confidence threshold
    results = model(frame, conf=conf) # Predict on the frame with confidence
threshold
    # Get the first result (since results is a list of results)
    result = results[0]
    # Render the predictions on the frame
    frame_with_detections = result.plot() # This will add the bounding boxes,
labels to the frame

    # Display the frame with the detected objects in an OpenCV window
    cv2.imshow("Video with Detections", frame_with_detections)

    # Press 'Esc' to exit
    if cv2.waitKey(1) & 0xFF == 27: # 1ms delay
        break

cap.release()
cv2.destroyAllWindows()

if __name__ == '__main__':
    # Choose either image or video
```

```
choice = input("Enter '1' for image or '2' for video: ")
if choice == '1':
    # Path to your image file
    image_path = "bai-tren(1) (1).jpg" # Replace with your image path
    predict_on_image(image_path, conf=0.2)
elif choice == '2':
    # Path to your video file
    video_path = "Vehicle Data - Dữ liệu xe cộ Việt Nam.mp4" # Replace with
your video path
    predict_on_video(video_path, conf=0.4)
else:
    print("Invalid choice. Please enter '1' for image or '2' for video.")
```

TÀI LIỆU THAM KHẢO

- [1] Glenn Jocher, et al. (2023). YOLOv8: State-of-the-art in real-time object detection.
- [2] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection.
- [3] Jocher, G., & Chia, H. (2022). YOLOv7: A Comprehensive Review of the State-of-the-art Real-time Object Detection. Journal of Computer Vision and Image Understanding, 13(4), 245-260.
- [4] Chien, H., & Lin, Y. (2023). Enhancing YOLO for Real-time Vehicle Detection in Traffic Monitoring. Journal of Traffic Technology, 11(2), 15-29.
- [5] Ultralytics Documentation. (2023). YOLOv8 User Guide. Ultralytics.
- [6] YOLOv8 Tutorial |How to Train YOLOv8 with Custom Dataset by Ultralytics.